

Atividade Docker + CI — [SEU NOME]

Preencha todos os campos marcados com [...] e substitua os prints de exemplo pelos seus. Salve as imagens em docs/imagens/ e mantenha os nomes de arquivo indicados.

Aluno(a): [nome completo] **Turma:** [turma] **Data:** [data] **Aplicação usada:**
[docker/getting-started-app — To-Do em Node.js]

1. Como executar este projeto

```
git clone [URL do seu repositório]
cd [pasta]
cp .env.example .env
docker compose up -d --build
```

Acesse: <http://localhost:3000>

Para derrubar: `docker compose down` (mantém dados) ou `docker compose down -v` (apaga dados).

2. Imagem e Dockerfile multi-stage

Estágios utilizados: [ex.: `builder` (instala dependências) e estágio final (runtime enxuto)]

Imagem base: [ex.: `node:20-alpine`] **Usuário de execução:** [ex.: `node`, `não-root`] **Tamanho final da imagem:** [ex.: 180MB]

Por que o multi-stage ajuda? [Sua resposta em 1–2 frases]

Print 1 — build + `docker images`

Print 2 — aplicação rodando com tarefas cadastradas

3. Volumes e persistência

Volume usado: [nome] → montado em [caminho dentro do container]

Print 3 — SEM volume: dados perdidos ao recriar o container

Print 4 — COM volume: dados preservados

Diferença entre `docker compose down` e `docker compose down -v`: [Sua resposta em 1 frase]

4. Rede

Rede criada: [nome] Serviços conectados: [app e db] A porta do banco está exposta ao host? [Não — justifique em 1 frase]

Por que o app consegue chamar o host `mysql / db` sem saber o IP? [Sua resposta em 1 frase]

Print 5 — `docker network inspect`

Print 6 — dados dentro do MySQL (`select * from todo_items;`)

5. Docker Compose

Serviços: [app, db] Rede: [nome] · Volume: [nome] Healthcheck em: [db] · `depends_on` com: [condition: service_healthy] Variáveis sensíveis: carregadas via `.env` (não versionado). Modelo em `.env.example`.

Print 7 — `docker compose ps`

6. Integração Contínua (GitHub Actions)

Arquivo do workflow: `.github/workflows/ci.yml` Gatilhos: [push e pull_request] O que o pipeline faz:

1. [valida o compose]
2. [builda a imagem]
3. [sobe a stack]
4. [aguarda a app responder e testa criar uma tarefa via API]

5. [derruba a stack]

Print 8 — execução verde 

7. Quebra proposital do CI

O que eu quebrei: [descreva a alteração exata que você fez] **Erro que apareceu no log:** [cole a mensagem principal] **Como o CI reagiu:** [em qual step falhou e por quê] **Como eu corriji:** [o que foi alterado]

Link do Pull Request: [URL]

Print 9 — execução vermelha  **+ log do erro**

8. Dificuldades e aprendizados

[3 a 5 linhas: o que travou, como resolveu, o que ficou mais claro sobre containers depois da atividade]

9. Checklist de autoavaliação

- Dockerfile multi-stage funcionando
- `.dockerignore` presente
- Container não roda como root
- Volume nomeado + persistência demonstrada
- Rede nomeada + banco não exposto ao host
- `compose.yaml` sobe tudo com um comando
- `.env` no `.gitignore` e `.env.example` versionado
- CI verde
- PR com CI vermelho documentado
- Todos os 9 prints no README

